

Multimodal Embedding Dimensions for Text-to-Image Vector Search

Executive findings

For an MLE choosing an embedding size for **text → image retrieval**, the strongest public evidence does **not** support a general rule that 3072 dimensions materially outperform 1536 or 768. The better-supported conclusion is more nuanced: **when a model has been explicitly trained with Matryoshka Representation Learning (MRL), the first large reduction in dimensionality is often remarkably cheap, while aggressive compression eventually produces a clear retrieval penalty**. Crucially, this is a *within-model* observation; dimension count is not a meaningful quality metric across different model families. ¹

The most directly applicable published experiment I found is **Amazon Nova Multimodal Embeddings**, because Amazon reports the *same multimodal model* at 3072, 1024, 384 and 256 dimensions on text→image retrieval. Moving from **3072 → 1024 dimensions cuts raw float32 vector storage by 67% while costing only 1.0 point on TextCaps and 1.1 points on MSCOCO**. Compression to 384 dimensions is more visible, and 256 dimensions incurs a substantial loss. ²

Nova MME dimensions	TextCaps image retrieval	MSCOCO image retrieval	ViDoRe v2 visual-doc retrieval	Raw float32/vector
3072	88.9	76.7	58.7	12 KiB
1024	87.9	75.6	57.7	4 KiB
384	85.6	72.9	53.4	1.5 KiB
256	83.1	70.6	50.2	1 KiB
Loss, 3072→1024	-1.0	-1.1	-1.0	-67%
Loss, 3072→384	-3.3	-3.8	-5.3	-87.5%
Loss, 3072→256	-5.8	-6.1	-8.5	-91.7%

The metrics above are those used by Amazon's report: average Recall@1/5/10 for image retrieval and NDCG@5 for ViDoRe v2, so the absolute scores should not be mixed with benchmarks reporting Recall@1 alone. Amazon explicitly says the lower-dimensional Nova vectors are prefixes of the 3072-d representation learned using MRL. ²

That pattern is independently consistent with the open-weight **Jina CLIP v2** results. Its text→image Recall@5 changes almost imperceptibly from 1024 to 768, 512 and even 256 dimensions on several datasets; the serious deterioration begins at 128 and especially 64 dimensions. ³

For a real-estate image-search system, my starting hypothesis would therefore be 768–1024 dimensions, not 3072. I would use 1536/3072 only if a domain evaluation demonstrates a meaningful gain on fine-grained, compositional property queries. Conversely, I would not go straight to 256

dimensions simply because general benchmarks make it look inexpensive: visually rich retrieval is one of the areas where aggressive compression appears to hurt more. This recommendation is an inference from Nova's and Jina's within-model ablations, rather than a published real-estate benchmark. ⁴

The evidence situation for the two models of greatest interest is uneven:

Model	Multimodal text↔image?	Dimension choices	Published multimodal dimension ablation?	Evidence quality for the size question
Gemini Embedding 2	Yes	128–3072; Google recommends 768 / 1536 / 3072	No public table found	Strong model benchmarks; weak direct size comparison
Cohere Embed 4	Yes	256 / 512 / 1024 / 1536	No public vendor table found	Good API flexibility; weak direct size comparison
Amazon Nova MME	Yes	256 / 384 / 1024 / 3072	Yes	Best commercial evidence found
Jina CLIP v2	Yes	MRL at 64 / 128 / 256 / 512 / 768 / 1024	Yes	Best open-weight evidence found

Gemini's current API documentation says Embedding 2 defaults to 3072 dimensions, uses MRL, recommends **768, 1536 or 3072**, and automatically re-normalises vectors when a non-default dimensionality is requested. Google's model page describes a supported range of **128–3072 dimensions**. ⁵ Cohere documents MRL outputs of **256, 512, 1024 and 1536** for Embed v4.0. ⁶

The practical message is therefore:

Treat dimension as a deploy-time efficiency knob inside an MRL-trained model, not as a proxy for model intelligence. Evaluate model family first, then dimensionality.

Gemini Embedding 2 and Cohere Embed

Gemini Embedding 2

Gemini Embedding 2 is particularly interesting for the proposed use case because it is natively multimodal rather than a text model and separately aligned vision encoder. Google maps text, images, video, audio and documents into one shared embedding space and explicitly positions the model for cross-modal retrieval. Its technical paper describes a Gemini backbone with bidirectional attention, pooling and projection into a **3072-dimensional full representation**, trained with MRL losses that include **768- and 1536-dimensional prefixes**. ⁷

Google's current Gemini API documentation makes dimensionality directly configurable:

- default: **3072**

- recommended: **768, 1536, 3072**
- model-information range: **128–3072**
- truncated Gemini Embedding 2 vectors are **automatically L2-normalised**, which is important when using cosine similarity or an equivalent dot product over unit vectors. ⁵

That last point removes an easy experimental error. With an MRL model, taking a prefix changes its norm; comparing an unnormalised truncated vector against a normalised full vector can confound the apparent dimensionality effect. Gemini Embedding 2 handles this automatically through the API. ⁸

The model's absolute cross-modal retrieval performance is strong. Google's paper reports the following Recall@1 scores for text→image retrieval:

Dataset	Gemini Embedding 2 text→image R@1
MSCOCO	62.9
Flickr30K	89.1
DOCCI	93.4
TextCaps	89.6
Mean	80.5

For the reverse image→text direction the paper reports a 91.2 mean R@1, including 78.8 on MSCOCO, 97.4 on Flickr30K, 91.3 on DOCCI and 97.4 on TextCaps. Evaluation embeds the two sides separately and retrieves using similarity over the test set. ⁹

For the real-estate application, **DOCCI is arguably more informative than COCO**. Google's Gemini paper emphasises its performance on detailed-caption datasets such as DOCCI and TextCaps rather than only short object-centric captions. A property query like *“sunlit open-plan kitchen with a waterfall-edge stone island, pale timber cabinetry and black pendant lights”* is much closer conceptually to detailed-caption retrieval than to identifying a dominant COCO object class. That is an application inference, but it makes Gemini's strong DOCCI result particularly worth testing on the property corpus. ⁹

There is, however, a major evidence gap:

I found no public Gemini Embedding 2 text→image table comparing 3072 vs 1536 vs 768.

Google's current documentation does contain a dimension ablation, but the table is explicitly for the older **Gemini Embedding 001 text model**, using MTEB rather than multimodal image retrieval:

Gemini Embedding 001 dimensions	MTEB
2048	68.16
1536	68.17
768	67.99
512	67.55
256	66.19

Gemini Embedding 001 dimensions	MTEB
128	63.31

This is encouraging evidence for MRL generally—1536 and 768 are essentially tied with the larger vector on that text benchmark—but it should **not** be treated as evidence that Gemini Embedding 2 will lose the same amount on complex images. ⁸

Google's own recommendation of 768/1536/3072, combined with the Embedding 2 paper's MRL training at those scales, makes those three sizes the sensible experimental points rather than arbitrary cuts such as 1024. ¹⁰

Cohere Embed v4 and v3

Cohere Embed v4.0 is also genuinely multimodal. Cohere documents image and text embeddings in a common space, and v4 can additionally turn **mixed image+text content into one fused vector**. That latter capability is potentially valuable for property search because a listing image can be combined with reliable textual information—room type, suburb, architectural style, agent annotations or structured attributes—rather than forcing the image alone to encode everything. ¹¹

Embed v4's MRL output dimensions are:

256, 512, 1024 and 1536. ⁶

This gives a particularly clean experiment: generate exactly the same catalogue and query workload at all four sizes and measure quality/cost curves. Cohere also supports reduced numeric representations including int8 and binary embeddings, so **dimensionality and numerical precision are two separate compression axes** that should be evaluated independently. ¹²

Cohere's v3 family also supports images, but v4 is substantially more useful for this particular study because v4 exposes the explicit Matryoshka `output_dimension` choices and supports mixed text+image inputs. ¹²

As with Gemini, I did **not** find a Cohere-published image-retrieval table giving TextCaps/MSCOCO performance separately at 1536, 1024, 512 and 256 dimensions. Cohere's documentation describes the available sizes and efficiency features but does not provide the dimension-vs-retrieval curve needed to answer the question directly. ⁶

There is one useful external datapoint, although it needs careful interpretation. Amazon's Nova technical report evaluated Cohere Embed 4 through the public Bedrock API and reports:

Same Amazon evaluation	Nova MME	Cohere Embed 4
MSCOCO image retrieval, avg R@1/5/10	76.7	22.9
TextCaps image retrieval, avg R@1/5/10	88.9	69.7
ViDoRe v2, NDCG@5	58.7	53.6

Amazon states that it ran Cohere through the public Bedrock API; these are therefore not Cohere self-reported numbers. They are valuable as a reproducible-ish commercial comparison, but Amazon is also

a direct competitor, and its report does not state the Embed 4 output dimension used for those rows. I would therefore treat these results as a reason to benchmark Cohere carefully, **not** as evidence about Cohere's 1536→1024→512→256 dimensionality curve. ¹³

For this application, Cohere's most compelling differentiator may consequently be less "1536 dimensions" and more **mixed-modality indexing**. Cohere itself highlights production search over product images plus multifaceted textual descriptions, which is structurally similar to real-estate listings even though it is not a controlled benchmark. ¹⁴

The strongest dimension-quality evidence

Amazon Nova gives the clearest commercial answer

Nova's ablation is unusually valuable because architecture, training and benchmark are held constant; only the prefix length changes. Amazon says its 1024-, 384- and 256-dimensional vectors can be obtained from prefixes of the 3072 representation because of MRL training. ²

The curve is informative:

3072 → 1024 is cheap. On the two image datasets, 67% fewer dimensions cost only 1.0–1.1 absolute retrieval points. On ViDoRe v2 the loss is likewise only 1.0 point. ¹⁵

1024 → 384 starts to matter. Relative to 3072, TextCaps loses 3.3 points, MSCOCO 3.8, and ViDoRe 5.3. The richer visual-document workload degrades faster than ordinary image retrieval. ¹⁵

256 is no longer close to free. Compared with 3072, TextCaps loses 5.8 points, MSCOCO 6.1 and ViDoRe 8.5. In relative terms, those are approximately 6.5%, 8.0% and 14.5% reductions respectively, calculated from Amazon's reported scores. ¹⁵

This supports a useful engineering hypothesis: **fine-grained multimodal information has a capacity floor**. MRL can strongly order information so that a relatively small prefix retains most retrieval signal, but eventually visual distinctions disappear. The fact that ViDoRe deteriorates more steeply than ordinary image retrieval is particularly relevant to complex imagery, although document screenshots are not the same distribution as property photography. ¹⁵

Jina CLIP v2 shows the same phenomenon at smaller scales

Jina CLIP v2 provides an especially useful open-weight result because it trains its 1024-dimensional representation with MRL objectives at **1024, 768, 512, 256, 128 and 64 dimensions** and publishes the cross-modal results at each length. ¹⁶

Text→image Recall@5 is:

Dimensions	CLIP Benchmark	Crossmodal-3600	XTD10
1024	79.10	81.43	84.87
768	79.12	82.35	84.85
512	78.93	82.31	84.60

Dimensions	CLIP Benchmark	Crossmodal-3600	XTD10
256	78.32	81.75	84.32
128	75.90	78.17	81.80
64	70.51	72.52	77.85

The striking result is that **768 is not worse than 1024 in practice**, and on one benchmark is actually modestly higher. Even 256 dimensions is within roughly one absolute point of 1024 across these three tests. The clear break occurs below that: 128 loses around 3–4 points and 64 around 7–9 points. ³

The reverse image→text results behave similarly. At 1024/768/512/256 dimensions the CLIP Benchmark Recall@5 scores are 89.73/89.60/89.55/89.35, while Crossmodal-3600 remains 83.23/83.26/83.21/82.81. Again, large dimensional reductions have little effect until the representation becomes very short. ³

This is good evidence against the intuition that “twice as many embedding dimensions means a substantially richer search vector”. For an MRL-trained model, the dimensions are **not equally informative independent coordinates**; training explicitly concentrates a useful representation into early prefixes. ¹⁷

It also explains why cross-model comparisons such as “Gemini is 3072-d whereas model X is 768-d” are misleading. Gemini Embedding 2 at 768 is still generated by the *same underlying Gemini embedding network*; the shortened output is not equivalent to replacing it with a smaller 768-dimensional vision-language architecture. ¹⁸

The storage and ANN incentive is large

For uncompressed float32 vectors, raw vector payload scales exactly with dimensionality:

Dimensions	Bytes/vector	Raw vectors at 10 million items	Relative to 3072
3072	12,288 B / 12 KiB	122.88 GB	100%
1536	6 KiB	61.44 GB	50%
1024	4 KiB	40.96 GB	33%
768	3 KiB	30.72 GB	25%
512	2 KiB	20.48 GB	17%
384	1.5 KiB	15.36 GB	12.5%
256	1 KiB	10.24 GB	8.3%

These are arithmetic payload sizes only; an actual ANN index also contains graph/product-quantisation structures, IDs, metadata and implementation overhead.

For a 10-million-image catalogue, **Gemini 768 versus 3072 saves roughly 92 GB of raw float32 vectors before indexing overhead**, while still using one of Google's explicitly recommended MRL sizes.

⁸ Distance computation and memory traffic also shrink with dimension as a first-order effect, although actual HNSW/IVF/DiskANN query latency must be measured in the target vector database because graph traversal, cache behaviour, quantisation and I/O can dominate.

That makes dimensionality reduction unusually attractive when its measured recall loss is under a point or two.

Image resolution, query complexity and real-estate imagery

Embedding length is only one compression stage. **Image preprocessing can destroy information before the vector dimension ever becomes relevant.**

Cohere provides unusually explicit image limits. For Embed v4, images above **2,458,624 pixels—1568×1568 for a square image—are downsampled**, while inputs smaller than 3,136 pixels (56×56) are upsampled. Its API accepts JPEG, PNG, WebP and GIF, with v4 supporting multiple images and a 20 MB `inputs` payload limit. ¹²

Gemini Embedding 2's current API documentation permits up to **six images per request** and documents PNG/JPEG support, but the embedding guide does not publish an equivalent explicit pixel downsampling threshold. It also supports up to 8,192 input tokens overall. ⁸

Jina CLIP v2 is informative here because its paper separately ablates **image resolution**. The model uses 512×512 image inputs in its final configuration, with training progressively increasing resolution. In the authors' ViDoRe experiment, moving from **224→384 pixels increased average NDCG@5 from 0.256 to 0.454**; 512 pixels produced a further useful gain, whereas moving 512→768 increased image-patch count by **2.25× for only +0.019 NDCG@5**. The authors select 512×512 as the efficiency/quality compromise. ¹⁹

That experiment is visual-document retrieval rather than house photography, so the exact resolution optimum should not be transferred to property data. The broader result is nevertheless important: **input resolution can have a much larger quality effect than cutting a well-trained MRL embedding from 1024 to 768 or 512 dimensions**. In Jina's results, the latter costs almost nothing while insufficient image resolution can destroy a very large share of retrieval quality. ²⁰

For real estate I would explicitly test resolution sensitivity for queries involving small or localised features:

"gas cooktop beneath a concealed rangehood";
"herringbone timber flooring";
"frameless shower with niche";
"ocean glimpse through the left-hand window";
"ceiling-mounted ducted air-conditioning vent";
"stone splashback continuing behind open shelving".

These are substantially harder than *"bedroom"* or *"house with pool"*. A low-resolution preprocess can make them impossible regardless of whether the final vector has 768 or 3072 coordinates.

There is also an important distinction between **semantic completeness** and **fine-grained compositional correctness**. An embedding may know that an image contains a kitchen but rank it highly for *"white kitchen, black island"* when the image is actually a black kitchen with a white island. Standard COCO-style retrieval can underexpose this kind of failure. Google's inclusion of detailed-caption datasets such as DOCCI and TextCaps is therefore encouraging for Gemini Embedding 2, but it still does not substitute for a property-specific benchmark. ⁹

Query length deserves a separate test from vector length. Public dimension ablations I found generally aggregate across benchmark queries; I did **not** find a Gemini/Cohere experiment stratifying the dimension penalty by text-query length. For the intended workload, I would expect a useful test set to span:

Query class	Example	Why it matters
Short category	“butler’s pantry”	Basic visual concept alignment
Attribute pair	“bathroom with green tiles”	Object + attribute binding
Compositional	“white kitchen with black island and brass pendants”	Multiple simultaneous constraints
Spatial	“pool behind house with covered entertaining area”	Layout/relations
Fine detail	“induction cooktop flush mounted into stone”	Small local feature
Negative/exclusion	“living room without carpet”	Embeddings often struggle with negation
Long natural-language	“bright north-facing living room with timber floors, high ceilings and doors opening onto a landscaped courtyard”	Long-query information retention

The critical evaluation question is not whether longer text produces a “better similarity score”—raw cosine values are not comparable as an accuracy metric across models—but whether the correct images move upward in the ranking.

What I would deploy for a real-estate search evaluation

The evidence points to a **two-stage decision: choose the best embedding model at a reasonably generous dimension, then compress it until domain recall becomes unacceptable**. Reversing that order can lead to selecting an inferior model simply because it exposes more vector coordinates.

I would put the following configurations into the first serious bake-off:

Candidate	Dimensions to test	Why it belongs
Gemini Embedding 2	768, 1536, 3072	Excellent detailed text↔image published results; precisely Google’s recommended MRL sizes
Cohere Embed 4	512, 1024, 1536; optionally 256	Native multimodal plus image+text fusion; clean MRL API
Amazon Nova MME	384, 1024, 3072	Only major commercial model found with a published multimodal per-dimension ablation
Jina CLIP v2	256, 512, 768, 1024	Open-weight control/baseline with unusually good published dimension ablations

The model capabilities and dimension choices above come directly from the respective model documentation and technical reports. ²¹

For **Gemini**, I would expect 768 to be a very strong cost/quality candidate, but that expectation is extrapolated from Google's MRL design plus results from Nova/Jina, not demonstrated by a Gemini Embedding 2 multimodal ablation. I would specifically compare 768 against 1536 before spending much time on 3072. ¹⁸

For **Cohere**, I would make 1024 the likely middle configuration. Test 512 aggressively because Jina's results suggest this class of compression can be surprisingly cheap, but do not assume Cohere has the same curve. I would include 1536 as the quality ceiling and 256 as a "how far can we push it?" point. ²²

For **Nova**, the published evidence is strong enough that 1024 should probably be the default engineering candidate unless your own catalogue shows otherwise: it uses one-third the coordinates of 3072 while losing only about one absolute point on Amazon's image and visual-document benchmarks. ¹⁵

For **Jina CLIP v2**, 512–768 looks especially interesting. Its published text→image scores are nearly flat through this range, making it useful both as a deployable open-weight option and as a control for determining whether a commercial API is actually adding enough domain value to justify its operational constraints. ³

One vector per image versus enriched representations

For real-estate search, I would also test three indexing representations separately:

Image-only embedding. This is the cleanest measurement of genuine visual grounding and should be the primary benchmark.

Image + trusted property text. Cohere v4 can fuse image and text into a single representation; Gemini Embedding 2 also supports multimodal/interleaved inputs. This is attractive when the text adds information genuinely associated with the pictured room rather than generic listing copy. ²³

Image plus generated dense caption as separate searchable vectors. This can sometimes recover visual concepts that the direct image embedding underweights, while still preserving direct multimodal retrieval. It should be evaluated as an ensemble rather than silently replacing the image benchmark, because generated captions can omit or hallucinate details.

For a listing with 20–40 photos, there is another architectural issue that probably matters more than 768 vs 1536 dimensions: **indexing at photo level versus property level**. A single whole-listing vector can wash out rare but important visual evidence. For a query such as "*homes with terrazzo bathrooms*", the useful evidence may exist in one photograph out of thirty. Photo-level retrieval followed by property-level aggregation is therefore the safer starting architecture.

A reproducible benchmark and decision rule

A useful property benchmark does not need millions of human labels. For dimension selection, **a few hundred high-quality queries over a realistic corpus can be much more discriminative than a giant generic benchmark**, provided relevance judgements are sufficiently deep.

I would build roughly 300–1,000 queries across the categories above, with deliberate hard negatives from visually similar properties. Each query should have graded relevance rather than one “correct” photograph: for example 2 = clearly satisfies all requested attributes, 1 = partially satisfies them, 0 = does not. That permits **NDCG@10**, while Recall@K can measure whether at least one truly useful result is surfaced.

The experiment should hold everything except dimensionality constant:

```
for model in candidate_models:
    for dimension in model_supported_dimensions:
        embed the same catalogue
        embed exactly the same queries
        use the same similarity metric
        use exact search first
        compute Recall@1/5/10, NDCG@10, MRR
        bootstrap confidence intervals over queries
```

Use exact nearest-neighbour search for the dimension experiment first. Otherwise a lower-dimensional representation may appear better simply because the ANN index achieves higher search recall at the same HNSW/IVF settings. Once the embedding-quality curve is known, benchmark ANN separately at a fixed *index recall* target and measure memory, QPS and p95/p99 latency.

For each model × dimension, I would record:

Measure	Why
Recall@1 / @5 / @10	Straightforward “did the relevant property/image surface?”
NDCG@10	Handles multiple partially/fully relevant images
MRR	Sensitive to where the first strong result appears
Hard-negative win rate	Especially useful for attribute/spatial mistakes
Results by query complexity	Reveals whether dimension hurts long/compositional queries
Results by image resolution	Separates vector compression from visual preprocessing
Exact-search quality	Measures embedding itself
ANN recall at target latency	Measures production index
Bytes/image and bytes/property	Makes quality-cost choice explicit
Embed throughput and API cost	Captures indexing economics

The decision rule I would use is a **non-inferiority frontier** rather than “highest benchmark score wins”. Choose the smallest vector for which the quality difference from the model's maximum dimension is statistically and operationally negligible. For example, one could predefine that a candidate is acceptable if NDCG@10 is within 1% relative and Recall@10 within 0.5 percentage points of the full-dimensional representation on the overall set, with no material regression on the fine-detail/compositional subsets. Those thresholds are product decisions, not universal benchmark standards.

A particularly informative experiment for Gemini would be:

```
Gemini Embedding 2
3072 → reference
1536 → compare ranking overlap and relevance
768 → compare ranking overlap and relevance
```

```
Stratify deltas by:
  simple room/category
  colour/material attributes
  multiple attributes
  spatial relationships
  small objects/details
  long descriptive queries
```

Because the API automatically normalises Gemini Embedding 2's reduced vectors, cosine/dot-product retrieval is straightforward at these supported sizes. ⁸ For any model where embeddings are manually prefix-truncated rather than returned through an API that normalises them, the prefixes should be L2-normalised before cosine-equivalent comparison.

Likely outcome

Based on the best direct evidence available today, I would expect the Pareto frontier to look roughly like this—not as guaranteed scores, but as the **hypothesis to test**:

3072 → 1536: usually difficult to justify from retrieval quality alone unless the domain is unusually fine-grained.

1536/1024 → 768/512: often still a very attractive operating region for MRL models; Jina shows almost no loss here, and Nova's 3072→1024 result is similarly flat. ²⁴

384/256: viable when memory/latency is highly constrained, but increasingly likely to lose information on complex multimodal retrieval. Nova's visual-document loss at 256 is particularly cautionary. ¹⁵

128/64: likely too aggressive for nuanced property-image search unless the system has a reranker or other retrieval stage compensating for lost information; Jina's published text→image curve drops sharply here. ³

The strongest actionable conclusion is therefore **not “3072 is best” or “768 is enough”**. It is that modern MRL models have made 3–4× vector compression plausible with surprisingly little quality loss, but **Gemini Embedding 2 and Cohere Embed 4 still lack the public multimodal per-dimension evidence needed to assume that result for real-estate imagery**. Nova MME and Jina CLIP v2 provide strong evidence that the hypothesis is worth testing, and they give useful controls against which Gemini's 3072/1536/768 and Cohere's 1536/1024/512/256 curves can be measured. ²⁵

For the specific **text → complex real-estate image** workload, the evidence supports starting with **Gemini Embedding 2 at 768 and 1536, Cohere Embed 4 at 512/1024/1536, Nova at 1024, and Jina CLIP v2 at 512/768/1024**, while giving image-resolution preprocessing, compositional hard negatives and photo-level indexing at least as much attention as embedding dimensionality itself.

1 5 8 10 21 **Embeddings | Gemini API | Google AI for Developers**

[https://ai.google.dev/gemini-api/docs/embeddings?](https://ai.google.dev/gemini-api/docs/embeddings?utm_campaign=gdm&utm_content=&utm_medium=referral&utm_source=deepmind.google)

[utm_campaign=gdm&utm_content=&utm_medium=referral&utm_source=deepmind.google](https://ai.google.dev/gemini-api/docs/embeddings?utm_campaign=gdm&utm_content=&utm_medium=referral&utm_source=deepmind.google)

2 4 13 15 25 **cdn.amazon.science**

<https://cdn.amazon.science/ba/f2/d0af272848748a24ba6ba45af3a7/nova-mme-technical-report-14.pdf>

3 16 17 19 20 24 **jina-clip-v2: Multilingual Multimodal Embeddings for Text and Images**

<https://arxiv.org/html/2412.08802v2>

6 11 12 22 23 **Introduction to Embeddings at Cohere | Cohere**

<https://docs.cohere.com/docs/embeddings>

7 9 18 **Gemini Embedding 2: A Native Multimodal Embedding Model from Gemini**

<https://arxiv.org/abs/2605.27295>

14 **Embed | Secure AI Retrieval | Cohere**

<https://cohere.com/embed>